UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA · FACULTAD DE INGENIERÍA EN SISTEMAS

Aseguramiento de la Calidad del Software

Clase 8

Proceso Fundamental del Testing

+ Hooks y Suites de Prueba en Playwright

Curso 048 · Módulo II · Sábado 12 de septiembre de 2026

Objetivos de la sesión

1. **Aplicar** el proceso fundamental de testing: planificación, análisis, ejecución y cierre.
2. **Elaborar** un Plan de Aseguramiento de la Calidad (SQA Plan) básico.
3. **Organizar** tests en suites con beforeEach, afterEach, beforeAll y afterAll.
4. **Gestionar** el estado entre tests de forma eficiente.

ESTRUCTURA DE LA CLASE

30

minutos de teoría

60

minutos de laboratorio

PARTE TEÓRICA

El Proceso Fundamental del Testing

No es solo “ejecutar casos de prueba” — son 5 actividades estructuradas.



Ya lo veníamos haciendo: en la Clase 04 vimos una versión de 5 pasos de este mismo proceso. Hoy la ampliamos con un paso nuevo y explícito: los criterios de salida.

PARTE TEÓRICA

Ejemplo aplicado: el proceso en Sauce Demo

1-2. Planificación + Diseño

Qué se prueba: login, inventario, carrito, checkout, logout. Casos: credenciales válidas/inválidas, usuario bloqueado, 6 productos con precio e imagen.

3. Implementación y Ejecución

Los tests de las Clases 05-08 (assertions, POM, evidencias, hooks) — ya construidos y corriendo.

4. Criterios de Salida

100% de los tests críticos (login, inventario, checkout) pasando, en paralelo y en secuencial.

5. Cierre

Reporte HTML de Playwright + evidencias en evidencias/ + lecciones (p. ej., el usuario de rendimiento degradado).

PARTE TEÓRICA

Plan de Aseguramiento de la Calidad (SQA Plan)

Documenta cómo se va a gestionar la calidad en un proyecto específico. Estructura mínima:

1. Propósito y alcance
2. Documentación de referencia
3. Estándares y políticas a aplicar
4. Actividades de QA y sus responsables
5. Herramientas y tecnologías
6. Gestión de defectos (flujo de vida del bug)
7. Criterios de entrada y salida por fase
8. Métricas e indicadores
9. Gestión de riesgos de calidad

Métricas de Calidad

Tasa de detección	Bugs encontrados / funcionalidades probadas	`3 bugs / 10 features = 0.3`
Eficiencia de pruebas	Tests pasados / Tests ejecutados × 100	`45/50 × 100 = 90%`
Cobertura de requisitos	Requisitos cubiertos / Total requisitos × 100	`8/10 × 100 = 80%`
Tiempo de resolución	Días desde reporte hasta cierre del bug	`Promedio: 2.5 días`

PARTE PRÁCTICA · 60 MIN

Hooks y Suites de Prueba

Laboratorio 08 sobre https://www.saucedemo.com — organizar tests con beforeEach/afterEach

Hooks

beforeAll · afterAll
beforeEach · afterEach

Helper reutilizable

helpers/auth.ts
loginAs(page, usuario)

Paralelismo

test.describe.configure
({ mode: 'parallel' })

Los Hooks de Playwright

conceptos.ts

```
test.describe('Suite de pruebas', () => {
  // Se ejecuta UNA VEZ antes de todos los tests
  test.beforeAll(async () => { /* ... */ });

  // Se ejecuta UNA VEZ después de todos los tests
  test.afterAll(async () => { /* ... */ });

  // Se ejecuta ANTES DE CADA test
  test.beforeEach(async ({ page }) => { /* ... */ });

  // Se ejecuta DESPUÉS DE CADA test
  test.afterEach(async ({ page }, testInfo) => { /* ... */ });

  test('Test 1', async ({ page }) => { /* ... */ });
});
```

Crear el helper de autenticación

Un patrón común: separar el login en una función reutilizable.

helpers/auth.ts

```ts
import { Page } from '@playwright/test';

export async function loginAs(page: Page, username: string) {
  const passwords: Record<string, string> = {
    'standard_user': 'secret_sauce',
    'problem_user': 'secret_sauce',
    'performance_glitch_user': 'secret_sauce',
    'locked_out_user': 'secret_sauce',
  };
  await page.goto('https://www.saucedemo.com');
  await page.locator('#user-name').fill(username);
  await page.locator('#password').fill(passwords[username] ?? 'secret_sauce');
  await page.locator('#login-button').click();
}
```

Suite del inventario: imports y beforeEach (1/3)

tests/clase08.spec.ts

```
import { test, expect } from '@playwright/test';
import { loginAs } from '../helpers/auth';

test.describe('Clase 08 - Suite de inventario con hooks', () => {
  test.describe.configure({ mode: 'parallel' });

  // Antes de cada test: login automático
  test.beforeEach(async ({ page }) => {
    await loginAs(page, 'standard_user');
    // Verificar que llegamos al inventario
    await expect(page).toHaveURL(/inventory/);
  });
  // continúa en la siguiente diapositiva...
```

Suite del inventario: afterEach (2/3)

tests/clase08.spec.ts

```
// Después de cada test fallido: capturar evidencia
test.afterEach(async ({ page }, testInfo) => {
  if (testInfo.status !== testInfo.expectedStatus) {
    const nombreSeguro = testInfo.title
      .replace(/[^a-z0-9]/gi, '_').toLowerCase();
    try {
      await page.screenshot({
        path: `./evidencias/fallo-${nombreSeguro}.png`,
        fullPage: true
      });
      console.log(`Test fallido: ${testInfo.title} - Screenshot guardado`);
      // continúa en la siguiente diapositiva...
```

Suite del inventario: afterEach (3/3)

tests/clase08.spec.ts

```
      } catch (e) {
        // Si la página ya fue cerrada antes del afterEach, el screenshot fallará
        console.log('No se pudo capturar screenshot:', e);
      }
    }
  });
```

Cantidad de productos y precio visible

tests/clase08.spec.ts

```
test('El inventario muestra 6 productos', async ({ page }) => {
  const items = page.locator('.inventory_item');
  await expect(items).toHaveCount(6);
});
test('Todos los productos tienen precio visible', async ({ page }) => {
  const precios = page.locator('.inventory_item_price');
  const cantidad = await precios.count();
  for (let i = 0; i < cantidad; i++) {
    const precio = precios.nth(i);
    await expect(precio).toBeVisible();
    const textoPrecio = await precio.textContent();
    expect(textoPrecio).toMatch(/^\$\d+\.\d{2}$/);  // formato: $9.99
  }
  console.log(`Todos los ${cantidad} productos tienen precio en formato correcto`);
});
```

Todos los productos tienen imagen visible

tests/clase08.spec.ts

```
test('Todos los productos tienen imagen visible', async ({ page }) => {
  const imagenes = page.locator('.inventory_item img');
  const cantidad = await imagenes.count();

  for (let i = 0; i < cantidad; i++) {
    await expect(imagenes.nth(i)).toBeVisible();
    const src = await imagenes.nth(i).getAttribute('src');
    expect(src).not.toBeNull();
  }
  console.log(`${cantidad} imágenes verificadas`);
});
```

El menú de hamburguesa funciona

tests/clase08.spec.ts

```
test('El menú de hamburguesa funciona', async ({ page }) => {
  // Abrir menú
  await page.locator('#react-burger-menu-btn').click();
  await page.waitForSelector('.bm-menu', { state: 'visible' });

  // Verificar opciones del menú
  await expect(page.getByText('All Items')).toBeVisible();
  await expect(page.getByText('About')).toBeVisible();
  await expect(page.getByText('Logout')).toBeVisible();
  await expect(page.getByText('Reset App State')).toBeVisible();

  // Cerrar menú
  await page.locator('#react-burger-cross-btn').click();
  await page.waitForSelector('.bm-menu', { state: 'hidden' });
});
```

Logout funciona correctamente

tests/clase08.spec.ts

```
  test('Logout funciona correctamente', async ({ page }) => {
    // Abrir menú y hacer logout
    await page.locator('#react-burger-menu-btn').click();
    await page.getByText('Logout').click();

    // Verificar que regresamos al login
    await expect(page).toHaveURL('https://www.saucedemo.com/');
    await expect(page.locator('#login-button')).toBeVisible();
  });

});
```

El `});` final cierra el primer test.describe. La siguiente diapositiva abre una SEGUNDA suite, independiente de esta.

Segunda suite: checkout con usuario estándar

tests/clase08.spec.ts

```
// ======= SUITE SEPARADA: Comportamiento con diferentes usuarios =======
test.describe('Clase 08 - Comportamiento por tipo de usuario', () => {
  test.describe.configure({ mode: 'parallel' });

  test('Usuario estándar puede completar el checkout', async ({ page }) => {
    await loginAs(page, 'standard_user');

    await page.locator('.btn_inventory').first().click();
    await page.locator('.shopping_cart_link').click();
    await page.locator('[data-test="checkout"]').click();

    await expect(page).toHaveURL(/checkout-step-one/);
    console.log('Usuario estándar llegó al checkout');
  });
```

Usuario con rendimiento degradado

tests/clase08.spec.ts

```
  test('Usuario de rendimiento degrado experimenta lentitud',
    async ({ page }) => {
    // NOTA: performance_glitch_user tiene un delay artificial de ~5 segundos en el login.
    // Es comportamiento esperado de la aplicación de demostración, no un error del test.
    // El timeout global (30 000 ms) es suficiente para absorber ese delay.
    const inicio = Date.now();
    await loginAs(page, 'performance_glitch_user');
    const tiempoLogin = Date.now() - inicio;
    console.log(`Tiempo de login (glitch user): ${tiempoLogin}ms`);
    // Este usuario tiene un delay artificial — documentarlo como hallazgo
    expect(tiempoLogin).toBeGreaterThan(0);
    await expect(page).toHaveURL(/inventory/);
  });

});
```

Ejecutar con paralelismo

Windows PowerShell

```
PS> npx playwright test tests/clase08.spec.ts
PS> npx playwright test tests/clase08.spec.ts --workers=1
PS> npx playwright show-report
```

Tiempos reales medidos (7 tests, 3 corridas cada modo):

~7-8 segundos en paralelo (con test.describe.configure({ mode: 'parallel' }))

~15 segundos con --workers=1 (secuencial, un test a la vez)

Ejercicio de diseño: SQA Plan mínimo

Qué hacer, paso a paso:

1. Formen parejas. Este ejercicio se hace en equipos de 2.

2. Tomen papel o un documento simple (no es el código del laboratorio) y respondan estos 4 puntos para Sauce Demo:

Propósito ¿para qué se prueba? (p. ej., verificar funciones críticas antes de un deploy)

Alcance ¿qué se prueba y qué NO se prueba?

Herramientas ¿con qué se prueba? (Playwright + TypeScript)

Criterios de salida ¿cuándo se considera listo? (p. ej., 100% de los tests críticos pasando)

3-4. Fotografíen o escaneen el resultado y súbanlo como documentos/sqa-plan-saucedemo.md junto con tu código — una oración breve por punto es suficiente, no hace falta un documento extenso.

TAREA DE LA CLASE 08

Tests reto: hooks y suites avanzados

Agrega estos 3 tests reto a tests/tarea08.spec.ts, con técnicas nuevas no vistas hoy.

Reto 1 — Suite serial con página compartida

test.describe.configure({ mode: 'serial' }) + una page creada en beforeAll y reutilizada entre tests (el modo serial por sí solo NO comparte la page).

Reto 2 — test.slow()

Marca el test del usuario con lentitud artificial como 'slow' para triplicar su timeout, en vez de confiar en el margen global.

Reto 3 — test.skip() dinámico

Omite un test en tiempo de ejecución según una condición evaluada dentro del propio test, documentando la razón.

Entrega: viernes 18 de septiembre, 23:59 (antes de la Clase 09).

PARA REFORZAR

Recursos complementarios

- **Planeación y Control de Pruebas (video)**

  https://www.youtube.com/watch?v=QG6EKXG3UF8

- **Playwright — Test hooks (documentación oficial)**

  https://playwright.dev/docs/api/class-test

- **SQA Plan — ejemplo de referencia**

  https://unpsjb.github.io/ids3t/index.html

La Prueba del Software como Ciencia — handbook.usfx.bo

CIERRE DE LA CLASE 8

Entregable de hoy

Sube a tu repositorio Git:

1. Archivo **`helpers/auth.ts`** y **`tests/clase08.spec.ts`** ejecutándose correctamente + los 3 tests reto.
2. Archivo **`documentos/sqa-plan-saucedemo.md`** con el SQA Plan mínimo.

Entrega en Canvas: enlace de tu repositorio (con helpers/, tests/ y documentos/) — viernes 18 de septiembre, 23:59.

Valor: **1 pt**

Próxima clase: Clase 9 — Automatización de Pruebas + Fixtures y Datos Parametrizados en Playwright.